HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 04

HỌC PHẦN: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên	**: PGS.TS. Trần Đình Quế**
Nhóm học phần	**: 05**
Họ và tên	**: Nguyễn Đức Trung**
Mã sinh viên	**: B23DCCN857**
Lớp	**: D23CNPM01**

HÀ NỘI 2026

MỤC LỤC

DANH MỤC HÌNH VẼ **4**

DANH MỤC CÁC THUẬT NGỮ **6**

1. TỔNG QUAN BÀI TOÁN **7**

2. YÊU CẦU VÀ ĐỊNH HƯỚNG THỰC HIỆN **8**

3. CƠ SỞ LÝ THUYẾT **8**

3.1. DEEP LEARNING DƯỚI GÓC NHÌN HÀM **8**

3.2. CONVOLUTION VÀ WEIGHT SHARING **9**

3.3. RELU, POOLING VÀ KÍCH THƯỚC **10**

3.4. TRAINING VÀ BACKPROPAGATION **10**

3.5. CLASSIFICATION VÀ REGRESSION **11**

4. MÔI TRƯỜNG THỰC NGHIỆM VÀ CẤU TRÚC NOTEBOOK **11**

5. THỰC NGHIỆM VỚI DATASET ẢNH **12**

5.1. MNIST **12**

5.1.1. CNN FROM SCRATCH BẰNG NUMPY **13**

5.1.2. BASIC CNN BẰNG PYTORCH **14**

5.1.3. ĐÁNH GIÁ BASIC CNN **15**

5.1.4. IMPROVED CNN VÀ BATCH NORMALIZATION **17**

5.1.5. TENSORFLOW/KERAS **17**

5.2. FASHION-MNIST **18**

5.2.1. CHUẨN BỊ DỮ LIỆU **18**

5.2.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ **19**

5.3. CIFAR-10 ..... **19**

5.3.1. DỮ LIỆU ẢNH MÀU **19**

5.3.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ **20**

5.4. SO SÁNH BA DATASET ẢNH **22**

6. THỰC NGHIỆM VỚI 3 DATASET KHÁC **24**

6.1. DIABETES **24**

6.1.1. KHÁM PHÁ DỮ LIỆU **24**

6.1.2. PYTORCH CONV1D **25**

6.2. HOUSE PRICE 26
6.2.1. KHÁM PHÁ VÀ PREPROCESSING 26
6.2.2. PYTORCH CONV1D 28
6.2.3. ĐÁNH GIÁ VÀ DIỄN GIẢI METRIC 28
6.3. COMMENTS 29
6.3.1. KHÁM PHÁ DỮ LIỆU 29
6.3.2. TOKENIZATION VÀ EMBEDDING 30
6.3.3. PYTORCH CNN 31
6.4. TENSORFLOW – DIABETES 32
6.5. TENSORFLOW – HOUSE PRICE 33
6.6. TENSORFLOW – COMMENTS 34
6.7. SO SÁNH PYTORCH VÀ TENSORFLOW TRÊN BA DATASET 1D 35
7. GIẢI THÍCH CODE NOTEBOOK 37
7.1. IMPORT VÀ DATA PIPELINE 37
7.2. MODEL DEFINITION VÀ FORWARD 38
7.3. TRAINING LOOP 39
7.4. EVALUATION 41
8. TỔNG HỢP KẾT QUẢ TOÀN BỘ THỰC NGHIỆM 43
Kết quả Image CNN 43
Kết quả Tabular/Text CNN 43
9. PHÂN TÍCH KẾT QUẢ VÀ BÀI HỌC 44
10. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 44
11. KẾT LUẬN 45
PHỤ LỤC – DATASET VÀ SOURCE CODE 46
Dataset 46
Source code 46

DANH MỤC HÌNH VẼ

Hình 1.1 – Sơ đồ mục tiêu và phạm vi Assignment 04 7
Hình 3.1 – Function composition trong CNN 9
Hình 3.2 – Minh họa kernel trượt trên chuỗi X 9
Hình 3.3 – Training workflow 11
Hình 4.1 – Cấu trúc thư mục Assignment 04 12
Hình 5.1 – Một số mẫu MNIST 13
Hình 5.2 – Output của convolution/activation/pooling NumPy 14
Hình 5.3 – Training curve Basic CNN 15
Hình 5.4 – Code evaluate Basic CNN 16
Hình 5.5 – So sánh Basic CNN và Improved CNN 17
Hình 5.6 – TensorFlow model summary 18
Hình 5.7 – 10 lớp Fashion-MNIST 19
Hình 5.8 – Mẫu ảnh CIFAR-10 theo class 20
Hình 5.9 – Training history CIFAR-10 21
Hình 5.10 – Confusion matrix của mô hình CNN trên tập kiểm tra CIFAR-10 22
Hình 5.11 – Kết quả notebook so sánh ba dataset 23
Hình 5.12 – So sánh Test Accuracy của mô hình CNN trên ba bộ dữ liệu ảnh. 24
Hình 6.1 – df.head()/shape của Diabetes 25
Hình 6.2 – House Price df.head()/shape 27
Hình 6.3 – House Price Feature Preprocessing 27
Hình 6.4 – Actual vs Predicted House Price 29
Hình 6.5 – Comments df.head()/columns 30
Hình 6.6 – Pipeline Text → Token IDs → Embedding → Conv1D 31
Hình 6.7 – TensorFlow Diabetes model summary 32
Hình 6.8 – TensorFlow House Price model summary 34
Hình 6.9 – TensorFlow Comments model summary 35
Hình 6.10 – So sánh Classification Accuracy của PyTorch và TensorFlow trên Diabetes và Comments 36
Hình 6.11 – So sánh R^2 của PyTorch và TensorFlow trên bài toán dự đoán giá nhà. 36

Hình 7.1 – Import/load data 37
Hình 7.2 – Khai báo class Diabetes 38
Hình 7.3 – Khai báo class Comments 39
Hình 7.4 – Hàm train dùng cho Diabetes và House Price 40
Hình 7.5 – TensorFlow model.fit() 41
Hình 7.6 – Code evaluate classification 42
Hình 7.7 – Code evaluate regression 43

DANH MỤC CÁC THUẬT NGỮ

Thuật ngữ	**Giải thích**
CNN	Mạng neural dùng convolution để học pattern cục bộ.
Conv2D	Convolution trên dữ liệu hai chiều như ảnh.
Conv1D	Convolution trên chuỗi một chiều.
Kernel/Filter	Bộ trọng số nhỏ trượt trên input.
Weight sharing	Một kernel được tái sử dụng tại nhiều vị trí.
ReLU	Activation max(0,x).
Pooling	Giảm kích thước representation.
Embedding	Ánh xạ token ID thành vector liên tục.
Flatten	Chuyển tensor thành vector.
Dense/Linear	Lớp kết nối đầy đủ.
Epoch	Một lượt model học qua toàn bộ training set.
Batch	Một nhóm mẫu được xử lý trong một bước update.
Loss	Hàm đo mức sai khác giữa prediction và target.
Autograd	Cơ chế tự động tính gradient trong PyTorch.
MAE	Mean Absolute Error.
RMSE	Root Mean Squared Error.
R^2	Coefficient of determination.
Accuracy	Tỷ lệ dự đoán đúng trên tổng số mẫu.

1. TỔNG QUAN BÀI TOÁN

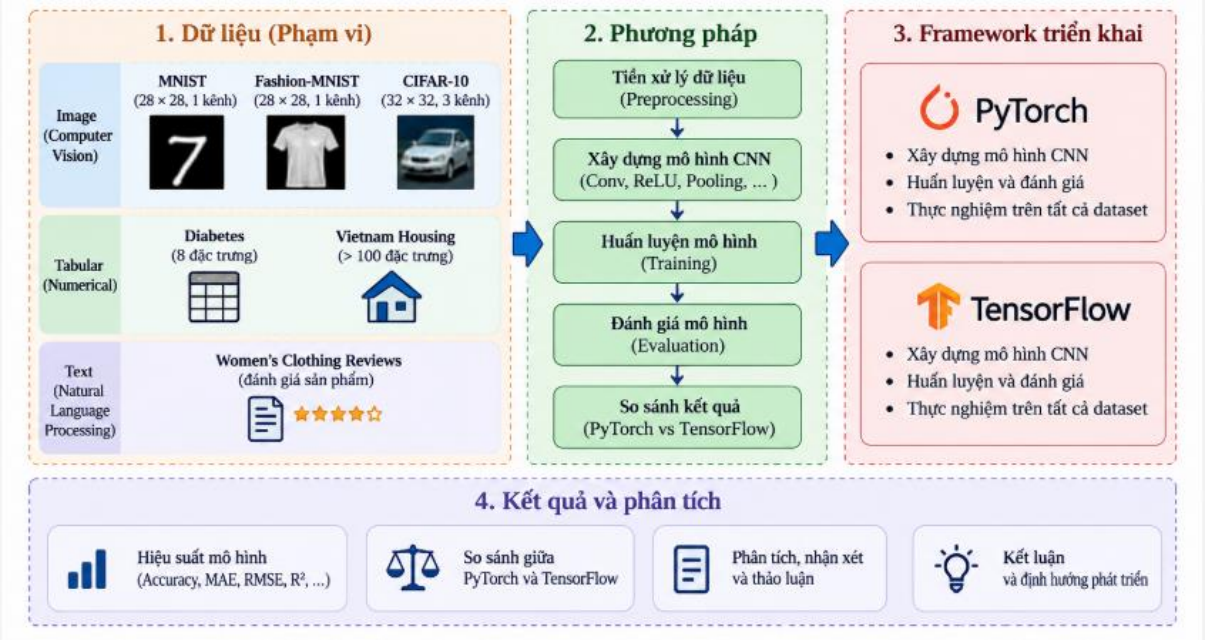
Assignment 04 tập trung vào Convolutional Neural Network (CNN) và cách tư duy CNN như một phép hợp thành các hàm. Nội dung thực nghiệm không chỉ nhằm đạt một con số accuracy cao mà nhằm chứng minh rằng cùng một ý tưởng Deep Learning có thể được điều chỉnh cho nhiều kiểu dữ liệu khác nhau.

Theo định hướng của bài học, ba loại dữ liệu chính được sử dụng trong phần thực nghiệm 1D gồm Diabetes dạng dữ liệu bảng, House Price dạng dữ liệu bảng và Comments dạng văn bản. Bên cạnh đó, notebook ảnh thực hiện CNN trên MNIST, Fashion-MNIST và CIFAR-10 để minh họa trường hợp CNN xử lý dữ liệu không gian hai chiều.

Hai notebook được xây dựng độc lập theo mục đích trình bày: notebook image_cnn_a04.ipynb tập trung vào CNN cho ảnh, NumPy convolution, Basic CNN, Improved CNN và TensorFlow; notebook tabular_text_cnn_a04.ipynb tập trung vào Conv1D cho Diabetes, House Price và Comments, sau đó cài đặt lại bằng TensorFlow.

Mục tiêu:

- Hiểu CNN như một chuỗi các phép biến đổi có tham số học được.
- Thực hiện phép convolution cơ bản bằng NumPy để quan sát cơ chế hoạt động.
- Xây dựng và huấn luyện CNN bằng PyTorch.
- Xây dựng CNN bằng TensorFlow/Keras.
- Phân biệt cách biểu diễn dữ liệu ảnh, dữ liệu bảng và dữ liệu văn bản trước khi đưa vào CNN.
- Đánh giá kết quả bằng các metric phù hợp với classification và regression.
- Phân tích sự khác nhau giữa các dataset và giữa các framework.



Hình 1.1 – Sơ đồ mục tiêu và phạm vi Assignment 04

Nhận xét: Điểm trọng tâm của bài là mối quan hệ giữa data representation, task và architecture, thay vì chỉ xem CNN là một lớp Conv2D/Conv1D có sẵn.

2. YÊU CẦU VÀ ĐỊNH HƯỚNG THỰC HIỆN

Yêu cầu thực hành được triển khai theo chuỗi: load dữ liệu, tìm hiểu kiến trúc CNN, xây dựng CNN từ scratch, xây dựng bằng PyTorch và xây dựng bằng TensorFlow. Trong phần ảnh, notebook đồng thời thực hiện so sánh Basic CNN và Improved CNN để minh họa tác động của thay đổi kiến trúc.

Đối với phần 1D, ba dataset được thống nhất là Diabetes, Vietnam Housing Dataset 2024 và Womens Clothing E-Commerce Reviews. Cách lựa chọn này giúp thể hiện ba trường hợp khác nhau: classification tabular, regression tabular và text classification.

Nhóm	Dataset	Dạng dữ liệu	Bài toán	Kiến trúc chính
Ảnh	MNIST	Ảnh xám	Classification	Conv2D
Ảnh	Fashion-MNIST	Ảnh xám	Classification	Conv2D
Ảnh	CIFAR-10	Ảnh màu	Classification	Conv2D
1D	Diabetes	Tabular	Binary classification	Conv1D
1D	House Price	Tabular	Regression	Conv1D
1D	Comments	Text	Binary classification	Embedding + Conv1D

Nhận xét: Việc giữ các dataset khác nhau trong cùng Assignment 04 cho phép quan sát rằng CNN không chỉ dành cho ảnh; điều quan trọng là thiết kế representation phù hợp.

3. CƠ SỞ LÝ THUYẾT

3.1. DEEP LEARNING DƯỚI GÓC NHÌN HÀM

Một mạng neural có thể được xem như một phép hợp thành các hàm. Với nhiều lớp, đầu ra của lớp trước trở thành đầu vào của lớp sau. Tư duy này dẫn tới biểu thức tổng quát $\hat{y} = f_L \circ \dots \circ f_2 \circ f_1(X)$. CNN vẫn tuân theo nguyên lý đó; điểm khác là một số hàm bên trong là phép convolution có cấu trúc cục bộ.

Trong bài học, một CNN được mô tả như sự kết hợp của convolution, activation, pooling hoặc downsampling và lớp phân loại/hồi quy. Các thành phần này tạo thành một representation hierarchy, tức là biểu diễn ngày càng phù hợp hơn với nhiệm vụ.

Neuron và hàm phi tuyến

Với một neuron tuyến tính, $z = w^T x + b$. Sau activation, $h = \sigma(w^T x + b)$. ReLU thường được viết $\text{ReLU}(x) = \max(0, x)$. Activation giúp mạng học quan hệ phi tuyến thay vì chỉ tạo ra một phép biến đổi tuyến tính liên tiếp.

CNN như function composition

Trong kiến trúc nhỏ của bài, có thể xem Conv1D là f1, Conv2D hoặc Conv1D tiếp theo kết hợp với ReLU là các phép biến đổi tiếp theo, còn Dense là lớp cuối. Vì vậy việc thay đổi architecture không làm mất bản chất Deep Learning: ta vẫn đang học các tham số để tối ưu một hàm mất mát.

$$X \rightarrow \text{Conv1D} \rightarrow \text{ReLU} \rightarrow \text{Conv1D} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \hat{y}$$

Hình 3.1 – Function composition trong CNN

Nhận xét: Cách nhìn CNN dưới dạng function composition giúp giải thích được code theo từng lớp thay vì chỉ mô tả tên model.

3.2. CONVOLUTION VÀ WEIGHT SHARING

Convolution trong CNN sử dụng một kernel nhỏ di chuyển trên dữ liệu. Kernel được tái sử dụng tại nhiều vị trí, vì vậy cùng một tập trọng số có thể phát hiện cùng một dạng pattern ở các vị trí khác nhau. Đây là cơ chế weight sharing.

Ví dụ trong bài học, X = [2, 1, 3, 4, 2] và W = [0.5, −1, 0.5]. Kernel đầu tiên tác động lên [2, 1, 3], sau đó dịch sang [1, 3, 4] và [3, 4, 2]. Ở vị trí đầu, giá trị convolution là $2(0.5) + 1(-1) + 3(0.5) = 1$.

Khái niệm	**Ý nghĩa**
Local connectivity	Mỗi kernel chỉ nhìn một vùng cục bộ tại một thời điểm.
Weight sharing	Một kernel được tái sử dụng trên nhiều vị trí.
Learned filter	Kernel được cập nhật qua backpropagation.
Feature map	Kết quả sau convolution biểu diễn mức đáp ứng của filter.

$$z_i = w_1 x_i + w_2 x_{i+1} + w_3 x_{i+2}$$

Hình 3.2 – Minh họa kernel trượt trên chuỗi X

Convolution tạo ra bias có cấu trúc: thay vì kết nối toàn bộ đầu vào như Dense, kernel tập trung vào các mẫu cục bộ và chia sẻ trọng số. Kernel có kích thước 3 được lần lượt áp dụng lên các vùng cục bộ của chuỗi đầu vào. Cùng một kernel được tái sử dụng tại nhiều vị trí khác nhau, thể hiện cơ chế chia sẻ trọng số của phép tích chập và giúp mô hình học các mẫu cục bộ một cách nhất quán.

3.3. RELU, POOLING VÀ KÍCH THƯỚC

Sau convolution, ReLU tạo phi tuyến. Trong notebook, ReLU được dùng trực tiếp sau Conv1D hoặc Conv2D. Điều này tạo ra các feature map có thể tiếp tục được xử lý bởi lớp convolution tiếp theo.

Pooling là phép giảm kích thước biểu diễn. Với max pooling kích thước 2, chuỗi [1, 5, 2, 4] trở thành [5, 4]. Pooling vừa giảm số lượng phần tử cần xử lý vừa làm representation ít nhạy hơn với những thay đổi vị trí nhỏ.

Kích thước đầu ra convolution

Với một chiều, kích thước đầu ra phụ thuộc vào kích thước input, kernel, padding và stride. Trong notebook, padding='same' được dùng ở nhiều lớp để giữ chiều dài trước khi pooling. Pooling mới là bước làm giảm resolution.

Thành phần	**Vai trò trong notebook**
Conv1D/Conv2D	Học pattern cục bộ.
ReLU	Thêm phi tuyến.
MaxPool	Giảm chiều không gian/chiều dài.
Adaptive/Global Pooling	Tạo representation kích thước ổn định.
Flatten	Chuyển tensor thành vector cho Dense.
Dense	Tạo output cuối.

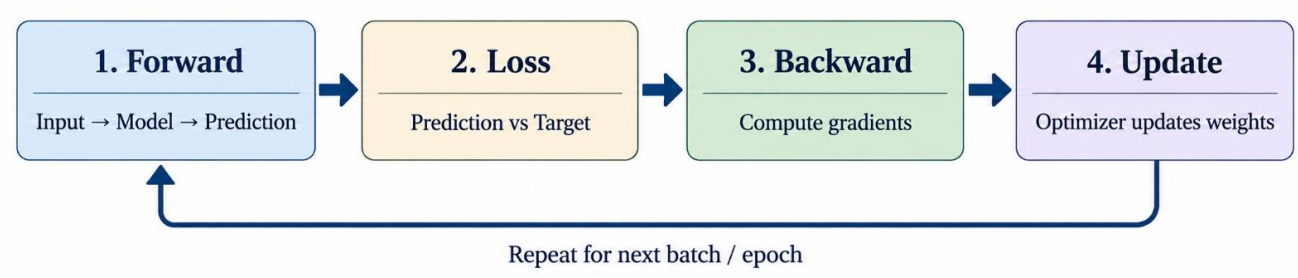
Khi đọc một CNN, cần theo dõi đồng thời kích thước không gian, số channel và vị trí của pooling để hiểu mạng đang giữ, giảm hay biến đổi thông tin.

3.4. TRAINING VÀ BACKPROPAGATION

Training CNN gồm các bước forward, loss, backward và update. Ở forward, dữ liệu đi qua các lớp để tạo prediction. Loss đo sai khác giữa prediction và target. Backward tính gradient của loss theo các tham số. Optimizer sau đó cập nhật tham số.

Trong PyTorch, quá trình này thể hiện rõ qua optimizer.zero_grad(), model(x), loss.backward() và optimizer.step(). Với TensorFlow/Keras, model.fit() quản lý phần lớn quá trình training và automatic differentiation.

Trong các notebook, Adam được sử dụng với learning rate 0.001. Số epoch chính được dùng là 5. Đây là cấu hình thực nghiệm giáo dục, nhằm quan sát quá trình học và so sánh architecture/framework hơn là tối ưu tuyệt đối.



Hình 3.3 – Training workflow

Quá trình huấn luyện CNN được thực hiện lặp lại qua bốn bước chính. Mô hình trước tiên thực hiện forward để tạo dự đoán, sau đó tính hàm mất mát dựa trên sai khác giữa dự đoán và nhãn thực tế. Tiếp theo, quá trình backward tính gradient của các tham số, và optimizer sử dụng các gradient này để cập nhật trọng số. Chu trình được lặp lại qua các batch và epoch cho đến khi mô hình đạt trạng thái hội tụ phù hợp.

3.5. CLASSIFICATION VÀ REGRESSION

Assignment 04 có cả classification và regression. Với binary classification, output thường dùng sigmoid để biểu diễn xác suất lớp 1. Trong PyTorch notebook, BCEWithLogitsLoss được dùng với output logits và sigmoid chỉ được áp dụng khi đánh giá.

House Price là regression nên output cuối là một giá trị liên tục, không dùng sigmoid. Loss MSE được dùng trong quá trình training; sau đó đánh giá bằng MAE, RMSE và R^2.

Bài toán	**Output**	**Loss/Metric chính**
Diabetes	Xác suất nhị phân	BCE / Accuracy
Comments	Xác suất nhị phân	BCE / Accuracy
House Price	Giá trị liên tục	MSE / MAE, RMSE, R^2

Việc chọn output theo task là một nguyên tắc quan trọng: architecture = data + task + representation. Nếu dùng output sai bản chất bài toán, việc huấn luyện và diễn giải kết quả sẽ không phù hợp.

Hai bài classification dùng accuracy để so sánh trực tiếp, trong khi House Price cần các metric regression vì sai số dự đoán là đại lượng liên tục.

4. MÔI TRƯỜNG THỰC NGHIỆM VÀ CẤU TRÚC NOTEBOOK

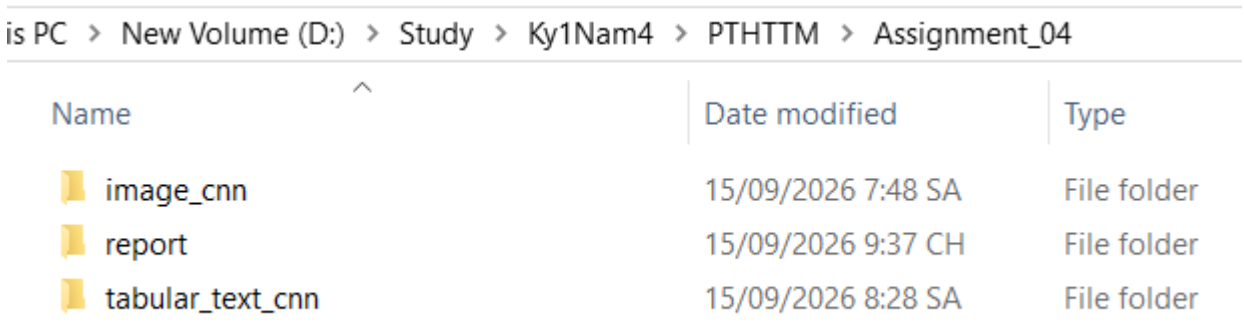
Thực nghiệm được thực hiện trong môi trường Python 3.11 có tên a04_cnn. Các thư viện chính gồm NumPy, Pandas, Matplotlib, Scikit-learn, PyTorch, Torchvision và TensorFlow.

Thành phần	**Vai trò**
Python 3.11	Môi trường chạy notebook

NumPy	Tính toán mảng và CNN from scratch
Pandas	Đọc và xử lý dữ liệu bảng
Matplotlib	Biểu diễn dữ liệu và training history
Scikit-learn	Split, preprocessing và evaluation
PyTorch	Xây dựng CNN và autograd
Torchvision	Dataset/image loading
TensorFlow/Keras	CNN với automatic differentiation và model.fit

Cấu trúc thư mục

Phần ảnh được tổ chức trong Assignment_04/image_cnn gồm data/mnist, data/fashion_mnist, data/cifar10 và notebook/image_cnn_a04.ipynb. Phần tabular/text nằm trong Assignment_04/tabular_text_cnn gồm data/diabetes, data/house_price, data/comments và notebook/tabular_text_cnn_a04.ipynb.



Hình 4.1 – Cấu trúc thư mục Assignment 04

Việc tách dữ liệu và notebook giúp đường dẫn tương đối rõ ràng và thuận tiện khi nộp hoặc đưa source code lên GitHub.

5. THỰC NGHIỆM VỚI DATASET ẢNH

5.1. MNIST

MNIST được sử dụng làm dataset ảnh chính để minh họa toàn bộ pipeline. Dữ liệu được lưu dưới dạng CSV, mỗi dòng gồm nhãn và 784 pixel tương ứng ảnh 28×28 grayscale. Notebook đọc mnist_train.csv và mnist_test.csv, tách label khỏi pixel, chuẩn hóa pixel về khoảng [0,1] và reshape về dạng (N, 1, 28, 28).

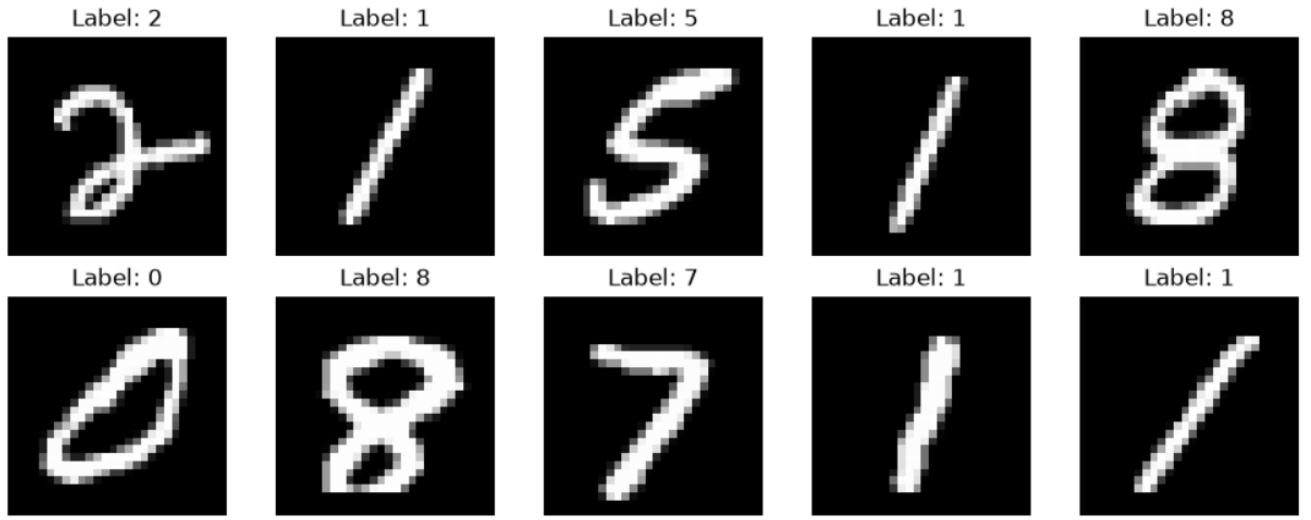
Tập	**Kích thước**

Train ban đầu	60,000 × 785
Test	10,000 × 785
Features	60,000 × 784
Image shape	1 × 28 × 28
Validation	6,000 mẫu
Train sau split	54,000 mẫu

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))

for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i].squeeze(), cmap="gray")
    ax.set_title(f"Label: {y_train[i]}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```



Hình 5.1 – Một số mẫu MNIST

5.1.1. CNN FROM SCRATCH BẰNG NUMPY

Notebook thực hiện các phép toán CNN cơ bản bằng NumPy trước khi dùng framework. Các hàm chính gồm convolution 2D một ảnh, ReLU và max pooling. Mục tiêu là làm rõ operation trước khi sử dụng nn.Conv2d hoặc Keras Conv2D.

Hàm convolution duyệt kernel trên từng vùng của ảnh, nhân từng phần tử tương ứng rồi cộng lại. ReLU được áp dụng sau convolution để tạo phi tuyến. Max pooling lấy giá trị lớn nhất trong từng vùng cục bộ.

Cách triển khai này chưa phải một hệ thống training CNN hoàn chỉnh với backpropagation đầy đủ; giá trị chính của phần scratch là giúp quan sát trực tiếp forward operation và liên hệ nó với các lớp framework.

```python
sample_image = X_train[0].squeeze()

kernel = np.array([
    [0.5, -1.0, 0.5],
    [0.5,  0.0, -0.5],
    [0.5,  1.0, 0.5]
])

conv_output = conv2d_single(sample_image, kernel)
relu_output = relu(conv_output)
pool_output = max_pool2d(relu_output)

print("Input:", sample_image.shape)
print("Convolution:", conv_output.shape)
print("ReLU:", relu_output.shape)
print("Pooling:", pool_output.shape)
```

```
Input: (28, 28)
Convolution: (26, 26)
ReLU: (26, 26)
Pooling: (13, 13)
```

Hình 5.2 – Output của convolution/activation/pooling NumPy

Nhận xét: Phần NumPy tạo cầu nối từ toán học sang API framework. Sau khi hiểu operation, việc chuyển sang nn.Conv2d trở nên dễ giải thích hơn.

5.1.2. BASIC CNN BẰNG PYTORCH

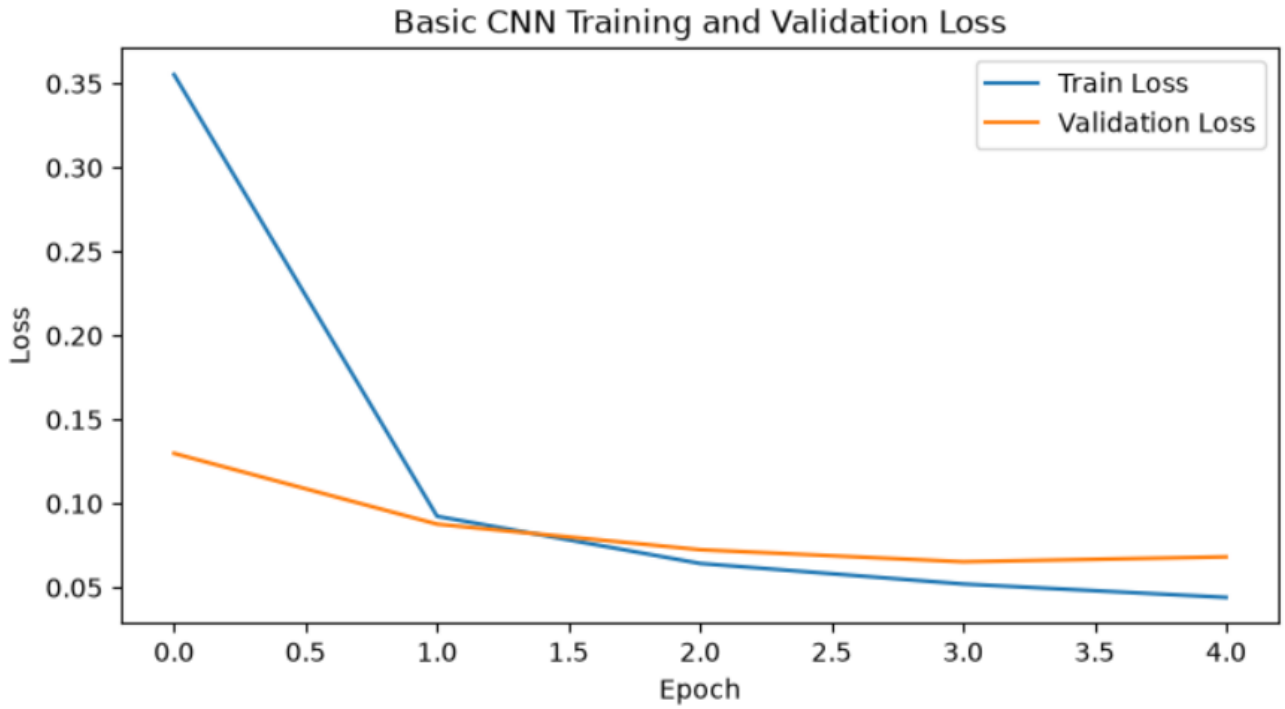
BasicCNN gồm hai block Conv2D → ReLU → MaxPool. Lớp đầu tiên chuyển từ 1 channel sang 16 channel; lớp thứ hai chuyển từ 16 sang 32 channel. Hai lần pooling giảm kích thước 28×28 xuống 7×7. Sau đó tensor 32×7×7 được Flatten và đưa vào Linear với 10 output.

Số tham số trainable của Basic CNN là 20,490. Model được huấn luyện 5 epoch.

Epoch	Train Acc	Validation Acc
1	0.8990	0.9602
2	0.9733	0.9742
3	0.9800	0.9807
4	0.9835	0.9785

5	0.9858	0.9808

Thời gian training ghi nhận là 137.15 giây. Test Accuracy đạt 0.9850.



Hình 5.3 – Training curve Basic CNN

Nhận xét: Biểu đồ cho thấy Training Loss giảm mạnh từ khoảng 0,36 xuống 0,045 qua 5 epoch, trong khi Validation Loss giảm từ khoảng 0,13 xuống 0,065. Sau epoch 3, Validation Loss gần như ổn định và có xu hướng tăng nhẹ ở epoch cuối. Nhìn chung, mô hình Basic CNN hội tụ nhanh và đạt Loss thấp, đồng thời chưa xuất hiện dấu hiệu overfitting nghiêm trọng trong số epoch được huấn luyện.

5.1.3. ĐÁNH GIÁ BASIC CNN

Báo cáo phân loại cho Basic CNN cho thấy performance khá đồng đều giữa các chữ số. Accuracy tổng thể đạt 0.9850 trên 10,000 ảnh test.

Class	**Precision**	**Recall**	**F1-score**
0	0.99	0.99	0.99
1	0.99	1.00	0.99
2	0.98	0.99	0.99
3	0.99	0.99	0.99
4	0.99	0.98	0.99
5	0.96	0.99	0.98

6	0.99	0.98	0.99
7	0.97	0.99	0.98
8	1.00	0.95	0.98
9	0.98	0.98	0.98

```python
def evaluate_model(model, test_loader):
    device = next(model.parameters()).device

    model.eval()

    predictions = []
    targets = []

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)

            outputs = model(images)
            preds = outputs.argmax(1).cpu().numpy()

            predictions.extend(preds)
            targets.extend(labels.numpy())

    accuracy = accuracy_score(targets, predictions)

    print(f"Test Accuracy: {accuracy:.4f}")
    print(classification_report(targets, predictions))

    return accuracy
```

```python
basic_accuracy = evaluate_model(
    basic_model,
    test_loader
)
```

Hình 5.4 – Code evaluate Basic CNN

Giải thích: Hàm evaluate_model() được sử dụng để đánh giá hiệu năng của mô hình trên tập kiểm tra. Hàm đưa mô hình về chế độ đánh giá, tắt tính toán gradient nhằm tiết kiệm tài nguyên, sau đó lần lượt đưa các batch dữ liệu vào mô hình để tạo dự đoán. Nhãn dự đoán và nhãn thực tế được lưu lại để tính Accuracy và các chỉ số Precision, Recall, F1-score thông qua Classification Report. Cuối cùng, hàm trả về độ chính xác trên tập test, được lưu vào biến basic_accuracy để phục vụ việc đánh giá mô hình Basic CNN.

Nhận xét: Các lớp có F1-score gần 0.98–0.99. Một số lỗi tương đối tập trung ở các chữ số có hình dạng dễ nhầm như 5, 7 và 8.

5.1.4. IMPROVED CNN VÀ BATCH NORMALIZATION

ImprovedCNN giữ cấu trúc cơ bản nhưng thêm Batch Normalization sau các lớp convolution. Số tham số tăng nhẹ từ 20,490 lên 20,586.

Model	Test Accuracy	Parameters	Training Time (s)
Basic CNN	0.9835	20,490	53.517681
Improved CNN	0.9867	20,586	69.745023

Improved CNN đạt 0.9855, cao hơn Basic CNN khoảng 0.0005 điểm accuracy, nhưng thời gian training tăng đáng kể. Vì vậy việc gọi model cải tiến không đồng nghĩa mọi metric đều cải thiện theo cùng một hướng.

	Model	Test Accuracy	Parameters	Training Time (s)
0	Basic CNN	0.9835	20490	53.517681
1	Improved CNN	0.9867	20586	69.745023

Hình 5.5 – So sánh Basic CNN và Improved CNN

Nhận xét: Bảng kết quả cho thấy Improved CNN đạt Test Accuracy cao hơn Basic CNN, lần lượt là 98,67% và 98,35%, tương ứng mức cải thiện khoảng 0,32 điểm phần trăm. Tuy nhiên, Improved CNN có số lượng tham số lớn hơn không đáng kể (20.586 so với 20.490) nhưng thời gian huấn luyện tăng từ 53,52 giây lên 69,75 giây. Nhìn chung, Improved CNN cho độ chính xác tốt hơn nhưng cần nhiều thời gian huấn luyện hơn, do đó có thể lựa chọn Improved CNN khi ưu tiên hiệu suất dự đoán.

5.1.5. TENSORFLOW/KERAS

TensorFlow model tái hiện ý tưởng Basic CNN bằng Keras Sequential. Kiến trúc gồm Conv2D 1→16, MaxPooling2D, Conv2D 16→32, MaxPooling2D, Flatten và Dense 10 với softmax.

Model có 20,490 tham số, tương đương Basic CNN về số lượng tham số. Sau 5 epoch, TensorFlow đạt Test Accuracy 0.9837.

Epoch	Train Acc	Val Acc	Val Loss
1	0.8970	0.9613	0.1330
2	0.9720	0.9730	0.0885
3	0.9798	0.9797	0.0726
4	0.9836	0.9820	0.0642

5	0.9862	0.9825	0.0605

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 14, 14, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten (Flatten)	(None, 1568)	0
dense (Dense)	(None, 10)	15,690

Total params: 20,490 (80.04 KB)

Trainable params: 20,490 (80.04 KB)

Non-trainable params: 0 (0.00 B)

Hình 5.6 – TensorFlow model summary

Nhận xét: TensorFlow đạt accuracy gần Basic CNN PyTorch. Sự chênh lệch nhỏ có thể xuất hiện do cách khởi tạo, thứ tự batch và chi tiết training khác nhau.

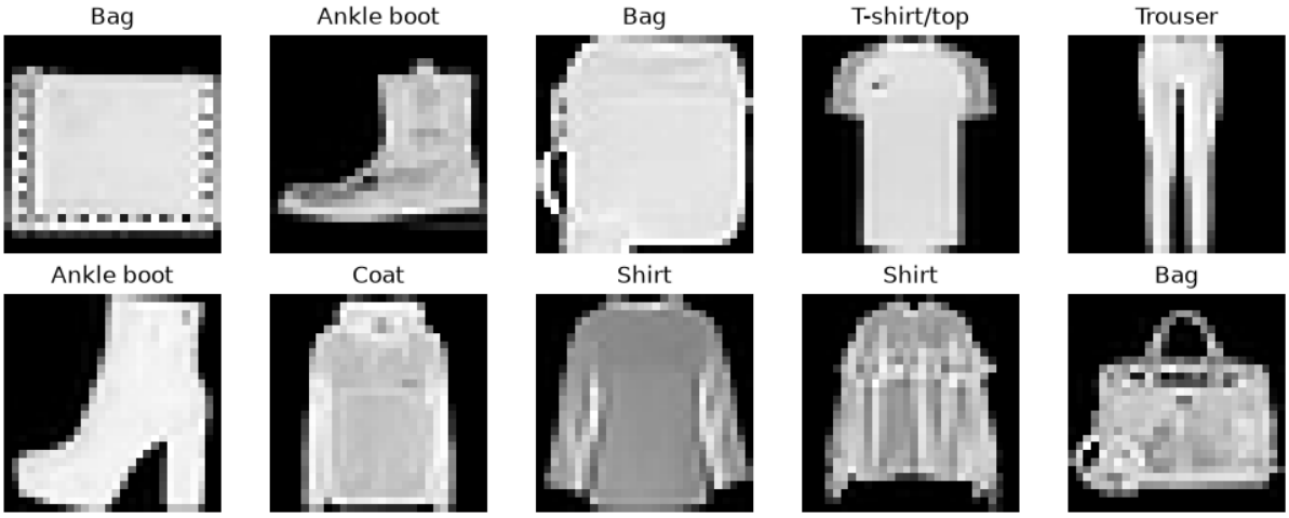
5.2. FASHION-MNIST

5.2.1. CHUẨN BỊ DỮ LIỆU

Fashion-MNIST có cùng kích thước ảnh 28×28 grayscale với MNIST nhưng nội dung khó hơn vì mỗi lớp là một loại sản phẩm thời trang. Dataset được đọc từ hai CSV, chuẩn hóa pixel, reshape về 1×28×28 và chia validation từ tập train.

Tập	**Shape**
Train	(60,000, 785)
Test	(10,000, 785)
Features	(60,000, 784)
Train sau split	(54,000, 1, 28, 28)

Validation	(6,000, 1, 28, 28)



Hình 5.7 – 10 lớp Fashion-MNIST

Nhận xét: So với MNIST, Fashion-MNIST có các class có hình thái gần nhau nên CNN cơ bản sẽ khó phân biệt hơn.

5.2.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ

Basic CNN được huấn luyện 5 epoch. Accuracy validation tăng từ 0.8502 ở epoch đầu lên 0.8920 ở epoch thứ năm. Test Accuracy cuối cùng là 0.9013.

Epoch	Train Acc	Val Acc
1	0.7798	0.8502
2	0.8627	0.8757
3	0.8798	0.8862
4	0.8893	0.8965
5	0.8956	0.8920

Training time là 53.91 giây.

Nhận xét: Accuracy 0.9013 thấp hơn MNIST rõ rệt, phản ánh độ khó của dữ liệu. Một số lớp có F1-score thấp hơn, đặc biệt những loại quần áo có hình ảnh tương tự.

5.3. CIFAR-10

5.3.1. DỮ LIỆU ẢNH MÀU

CIFAR-10 gồm ảnh màu 32×32 với 3 channel RGB và 10 lớp. Trong notebook, ImageFolder của torchvision được dùng để đọc thư mục train/test, mỗi class nằm trong một thư mục riêng.

Thuộc tính	Giá trị
Train samples	50,000
Test samples	10,000
Input	3 × 32 × 32
Classes	10
Train batches	391
Test batches	79



Hình 5.8 – Mẫu ảnh CIFAR-10 theo class

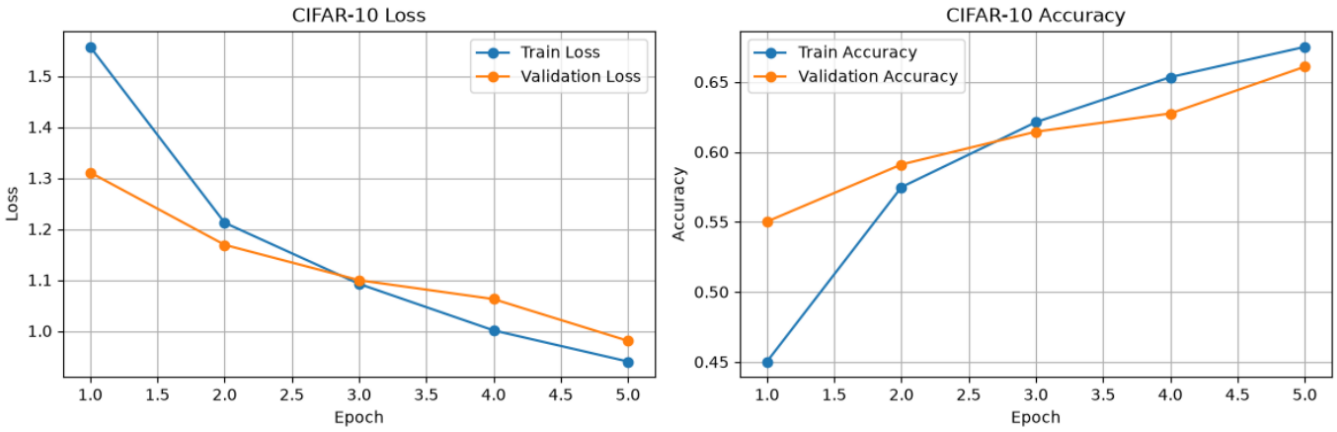
Nhận xét: CIFAR-10 khó hơn hai dataset grayscale vì ảnh màu có nhiều biến thiên về nền, hình dạng và texture; kiến trúc CNN đơn giản sẽ bị giới hạn.

5.3.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ

Model CIFAR-10 được huấn luyện 5 epoch. Train accuracy tăng từ 0.4160 lên 0.6572; validation accuracy tăng từ 0.5184 lên 0.6416. Test Accuracy đạt 0.6477.

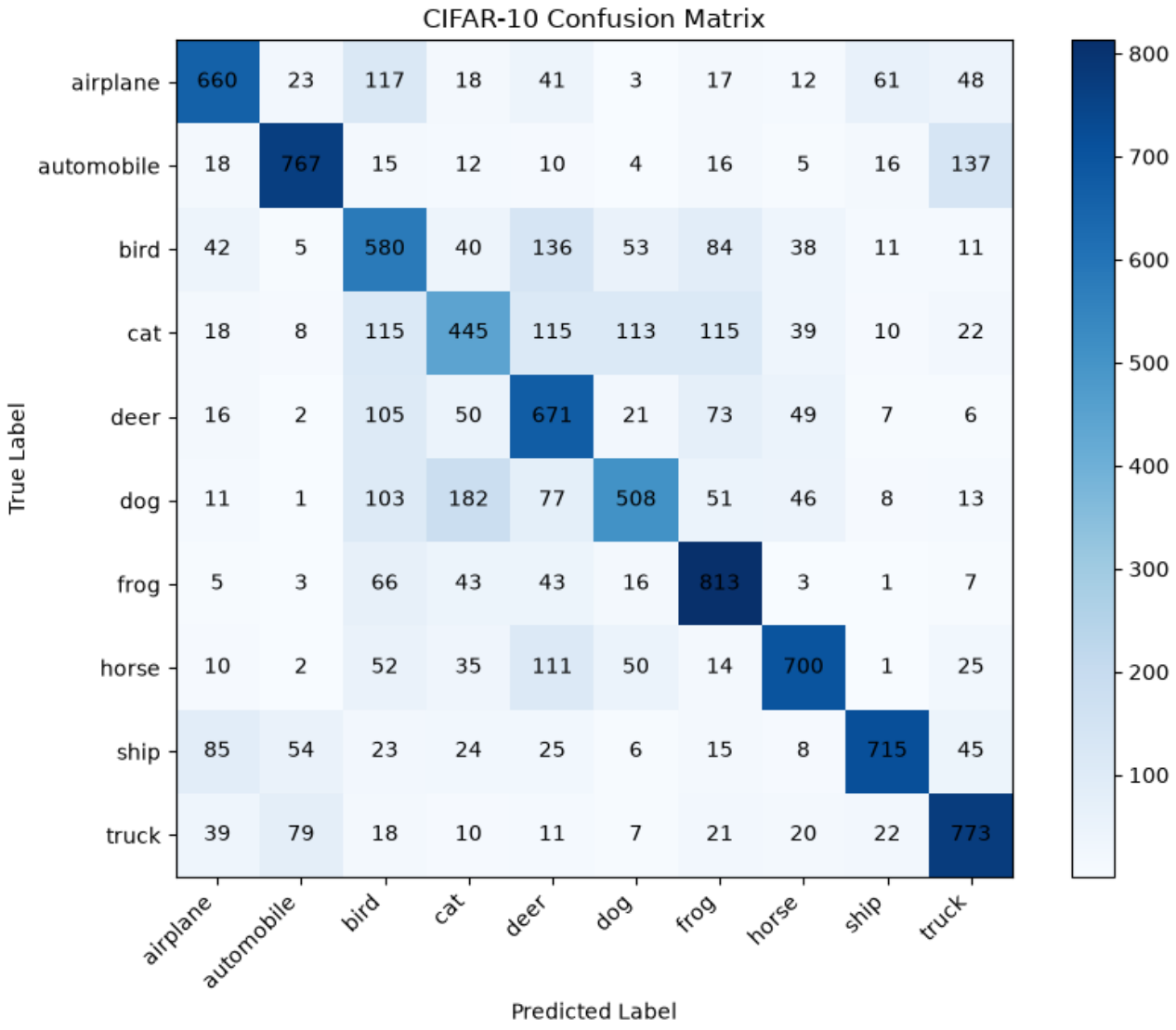
Epoch	Train Acc	Val Acc
1	0.4160	0.5184
2	0.5523	0.5732
3	0.5987	0.6062
4	0.6306	0.6256
5	0.6572	0.6416

Training time là 800.70 giây, lớn hơn đáng kể so với MNIST và Fashion-MNIST.



Hình 5.9 – Training history CIFAR-10

Nhận xét: Accuracy của cả tập training và validation tăng dần qua 5 epoch. Train accuracy tăng từ 0.4160 lên 0.6572, trong khi validation accuracy tăng từ 0.5184 lên 0.6416. Kết quả cho thấy mô hình tiếp tục học được đặc trưng của dữ liệu trong quá trình huấn luyện, nhưng mức accuracy cuối cùng vẫn còn hạn chế đối với CIFAR-10.



Hình 5.10 – Confusion matrix của mô hình CNN trên tập kiểm tra CIFAR-10

Nhận xét: Các giá trị trên đường chéo chính thể hiện số mẫu được phân loại đúng. Các giá trị ngoài đường chéo cho thấy những trường hợp mô hình nhầm giữa các lớp. Kết quả confusion matrix phù hợp với classification report, trong đó hiệu năng giữa các lớp không đồng đều. Test Accuracy tổng thể đạt 0.6477 trên 10.000 ảnh kiểm tra.

Accuracy 0.6477 cho thấy CNN cơ bản chưa đủ mạnh để xử lý tốt CIFAR-10 trong cấu hình 5 epoch. Đây không phải lỗi; kết quả thể hiện giới hạn của architecture và training budget.

5.4. SO SÁNH BA DATASET ẢNH

Dataset	**Input**	**Test Accuracy**	**Training Time (s)**
MNIST	1 × 28 × 28	0.9850	137.146
Fashion-MNIST	1 × 28 × 28	0.9013	53.912

CIFAR-10	3 × 32 × 32	0.6477	800.702

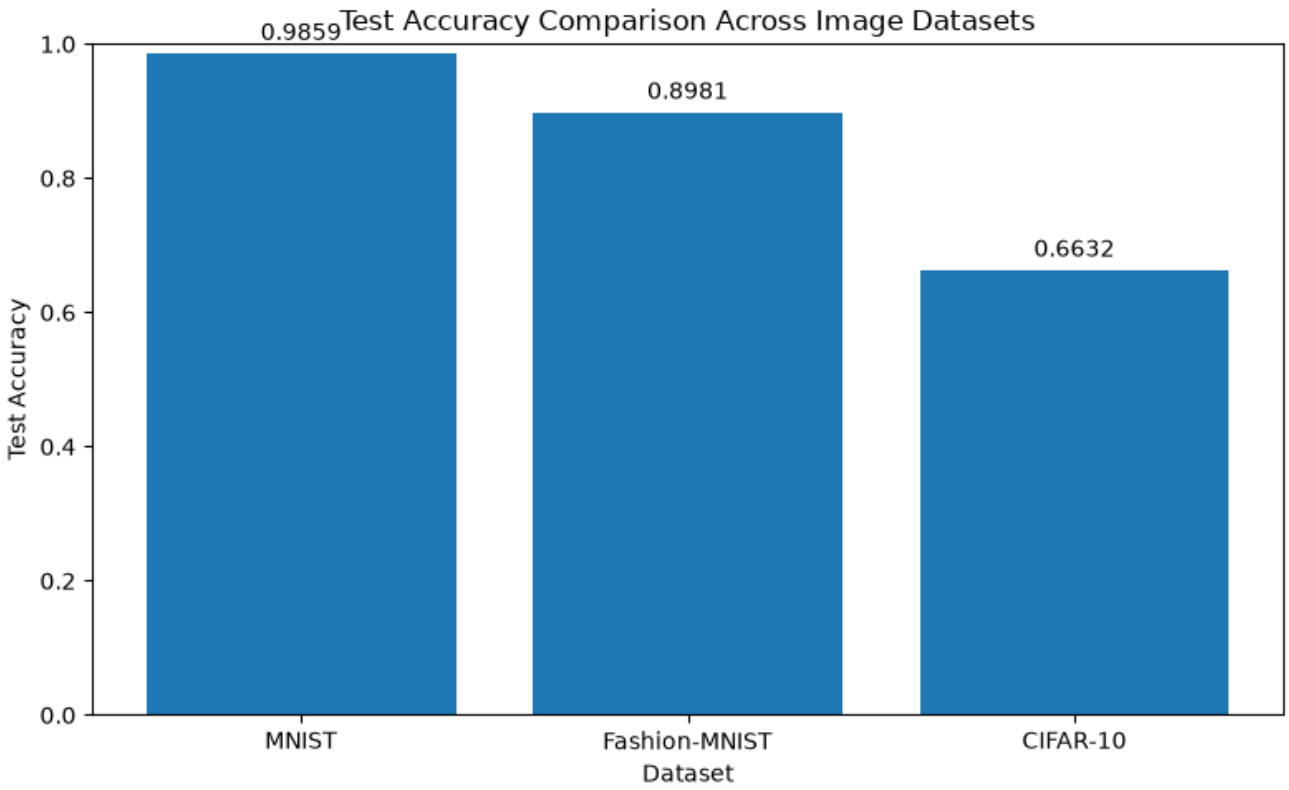
MNIST đạt kết quả cao nhất. Fashion-MNIST thấp hơn nhưng vẫn trên 90%. CIFAR-10 thấp nhất và có thời gian training dài nhất trong ba thực nghiệm.

```
image_comparison = pd.DataFrame({
    "Dataset": [
        "MNIST",
        "Fashion-MNIST",
        "CIFAR-10"
    ],
    "Input Shape": [
        "1 × 28 × 28",
        "1 × 28 × 28",
        "3 × 32 × 32"
    ],
    "Test Accuracy": [
        basic_accuracy,
        fashion_accuracy,
        cifar_accuracy
    ],
    "Training Time (s)": [
        basic_time,
        fashion_time,
        cifar_time
    ]
})

image_comparison
```

	Dataset	Input Shape	Test Accuracy	Training Time (s)
0	MNIST	1 × 28 × 28	0.9835	53.517681
1	Fashion-MNIST	1 × 28 × 28	0.8978	58.053910
2	CIFAR-10	3 × 32 × 32	0.6709	623.468082

Hình 5.11 – Kết quả notebook so sánh ba dataset



Hình 5.12 – So sánh Test Accuracy của mô hình CNN trên ba bộ dữ liệu ảnh.

Nhận xét: Mô hình CNN đạt Test Accuracy cao nhất trên MNIST với 0.9850, tiếp theo là Fashion-MNIST với 0.9013 và CIFAR-10 với 0.6477. Kết quả cho thấy cùng một kiến trúc CNN có hiệu năng khác nhau tùy thuộc vào đặc điểm và mức độ phức tạp của bộ dữ liệu. MNIST gồm ảnh chữ số viết tay đơn giản, trong khi CIFAR-10 gồm ảnh RGB với các đối tượng phức tạp hơn.

Độ khó của dữ liệu và kích thước representation ảnh hưởng rõ tới kết quả. CIFAR-10 có RGB và nội dung đa dạng hơn nên CNN đơn giản cần nhiều năng lực biểu diễn hơn.

6. THỰC NGHIỆM VỚI 3 DATASET KHÁC

6.1. DIABETES

6.1.1. KHÁM PHÁ DỮ LIỆU

Diabetes sử dụng file diabetes.csv từ bài trước, gồm 768 mẫu và 9 cột, trong đó Outcome là target. Các feature gồm Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction và Age.

Thông tin	**Giá trị**

Rows	768
Columns	9
Features sau preprocessing	8
Train	537
Validation	115
Test	116
Task	Binary classification

Pipeline preprocessing sử dụng median imputation và StandardScaler cho numeric features; categorical pipeline được chuẩn bị nhưng dataset thực tế không có categorical feature cần mã hóa. Dữ liệu sau preprocessing có shape (768, 8).

```
DIABETES_PATH = "../data/diabetes/diabetes.csv"

diabetes = pd.read_csv(DIABETES_PATH)

print("Shape:", diabetes.shape)
diabetes.head()
```

```
Shape: (768, 9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

Hình 6.1 – df.head()/shape của Diabetes

Nhận xét: Dataset nhỏ nên thời gian training rất ngắn. Với Conv1D, 8 feature được xem như một chuỗi feature một chiều.

6.1.2. PYTORCH CONV1D

DiabetesCNN gồm Conv1D 1→16, ReLU, Conv1D 16→16, ReLU, MaxPool1D và Dense output 1. Input được reshape thành (samples, 8, 1), sau đó transpose trong forward để PyTorch nhận dạng dạng (batch, channels, length).

Model được train 5 epoch bằng Adam và BCEWithLogitsLoss. Loss giảm từ 0.6996 xuống 0.5972.

Epoch	**Loss**
1	0.6996

2	0.6779
3	0.6529
4	0.6309
5	0.5972

Training time là 0.50 giây và Test Accuracy đạt 0.6983.

Nhận xét: Loss giảm đều nhưng accuracy test chỉ đạt khoảng 69.83%. Điều này cho thấy việc áp dụng Conv1D lên feature sequence tabular là một thí nghiệm kiến trúc, không đảm bảo vượt các mô hình tabular chuyên dụng.

6.2. HOUSE PRICE

6.2.1. KHÁM PHÁ VÀ PREPROCESSING

Vietnam Housing Dataset 2024 gồm 30,229 dòng và 12 cột. Target là Price. Các feature numeric gồm Area, Frontage, Access Road, Floors, Bedrooms và Bathrooms. Các categorical feature gồm House direction, Balcony direction, Legal status và Furniture state. Address được xử lý thêm để tạo thông tin Location.

Thông tin	**Giá trị**
Rows	30,229
Columns	12
Processed features	133
Train	21,160
Validation	4,534
Test	4,535
Task	Regression

Pipeline numeric sử dụng median imputation và StandardScaler; categorical sử dụng most-frequent imputation và OneHotEncoder(handle_unknown='ignore', sparse_output=False). Sau encoding, số feature tăng lên 133.

```
HOUSE_PATH = "../data/house_price/vietnam_housing_dataset.csv"

house = pd.read_csv(HOUSE_PATH)

print("Shape:", house.shape)
house.head()
```

```
Shape: (30229, 12)
```

	Address	Area	Frontage	Access Road	House direction	Balcony direction	Floors	Bedrooms	Bathrooms	Legal status	Furniture state	Price
0	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	84.0	NaN	NaN	NaN	NaN	4.0	NaN	NaN	Have certificate	NaN	8.60
1	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	60.0	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN	7.50
2	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	90.0	6.0	13.0	Đông - Bắc	Đông - Bắc	5.0	NaN	NaN	Sale contract	NaN	8.90
3	Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ C...	54.0	NaN	3.5	Tây - Nam	Tây - Nam	2.0	2.0	3.0	Have certificate	Full	5.35
4	Đường Quang Trung, Phường 8, Gò Vấp, Hồ Chí Minh	92.0	NaN	NaN	Đông - Nam	Đông - Nam	2.0	4.0	4.0	Have certificate	Full	6.90

Hình 6.2 – House Price df.head()/shape

Feature Preprocessing

Missing numerical values are filled with the median and standardized.

Missing categorical values are filled with the most frequent value and one-hot encoded.

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

house_preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_cols),
    ("cat", categorical_pipeline, categorical_cols)
])

X_house_processed = house_preprocessor.fit_transform(X_house)

print("Processed shape:", X_house_processed.shape)
```

```
Processed shape: (30229, 133)
```

Hình 6.3 – House Price Feature Preprocessing

Nhận xét: One-hot encoding làm representation tăng từ số feature gốc lên 133 chiều. Đây là lý do cần theo dõi shape trước khi thiết kế Conv1D.

6.2.2. PYTORCH CONV1D

HousePriceCNN dùng hai Conv1D 1→16 và 16→16, mỗi lớp đi kèm ReLU, sau đó MaxPool1D và AdaptiveAvgPool1D(1). Adaptive average pooling giúp output cuối có kích thước cố định, tránh phụ thuộc vào chiều dài 133 sau pooling.

Regressor gồm Flatten và Linear(16,1). Model được huấn luyện bằng MSELoss và Adam trong 5 epoch.

Epoch	**Training Loss**
1	11.8371
2	4.5061
3	4.1287
4	3.9243
5	3.8909

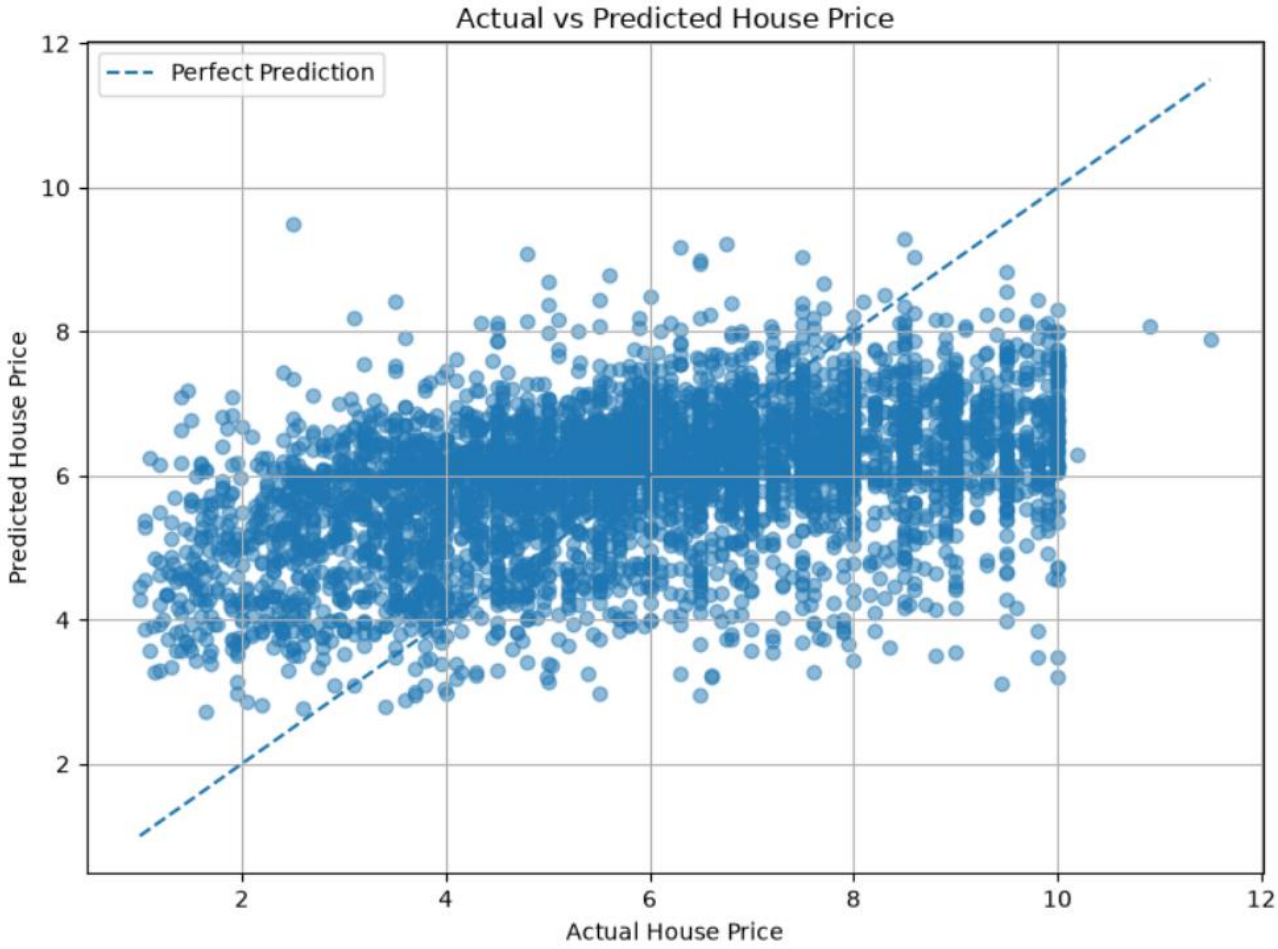
Training time là 6.67 giây. Test metrics: MAE = 1.6100, RMSE = 1.9684, R^2 = 0.1905.

Nhận xét: Loss giảm mạnh từ epoch 1 sang epoch 2 rồi ổn định dần. R^2 dương 0.1905 cho thấy model giải thích được một phần biến thiên giá nhưng representation CNN đơn giản vẫn còn hạn chế.

6.2.3. ĐÁNH GIÁ VÀ DIỄN GIẢI METRIC

MAE bằng 1.6100 nghĩa là sai số tuyệt đối trung bình giữa giá dự đoán và giá thực tế, theo đơn vị target của dataset. RMSE bằng 1.9684 lớn hơn MAE vì RMSE phạt mạnh hơn các sai số lớn. R^2 = 0.1905 cho thấy mức độ giải thích biến thiên của model còn khiêm tốn.

Metric	**Giá trị**	**Diễn giải**
MAE	1.6100	Sai số tuyệt đối trung bình
RMSE	1.9684	Căn bậc hai của MSE, nhạy với lỗi lớn
R^2	0.1905	Mức giải thích biến thiên của target



Hình 6.4 – Actual vs Predicted House Price

Nhận xét: Mỗi điểm trên biểu đồ biểu diễn mối quan hệ giữa giá nhà thực tế và giá nhà được mô hình CNN dự đoán. Đường nét đứt biểu diễn trường hợp dự đoán hoàn hảo, trong đó giá trị dự đoán bằng giá trị thực tế. Các điểm càng gần đường tham chiếu cho thấy dự đoán càng gần giá trị thực tế. Kết quả cho thấy các dự đoán có xu hướng bám theo giá trị thực tế nhưng vẫn tồn tại sai lệch ở một số mẫu, phù hợp với kết quả đánh giá của mô hình với MAE = 1.6100, RMSE = 1.9684 và R^2 = 0.1905.

6.3. COMMENTS

6.3.1. KHÁM PHÁ DỮ LIỆU

Dataset Womens Clothing E-Commerce Reviews có 23,486 dòng và 11 cột. Các cột bao gồm Clothing ID, Age, Title, Review Text, Rating, Recommended IND, Positive Feedback Count và các trường phân loại về Division, Department và Class.

Trong Assignment 04, Review Text được chọn làm input và Recommended IND làm target nhị phân. Sau khi loại các dòng thiếu Review Text hoặc target, còn 22,641 mẫu.

Thông tin	**Giá trị**

Rows ban đầu	23,486
Columns	11
Rows sau dropna	22,641
Positive Recommended	18,540
Negative Recommended	4,101
Train	15,848
Validation	3,396
Test	3,397

```
comments = pd.read_csv(
    os.path.join(COMMENTS_DIR, "womens_clothing_reviews.csv")
)

print("Shape:", comments.shape)
print("Columns:")
print(comments.columns.tolist())

comments.head()
```

```
Shape: (23486, 11)
Columns:
['Unnamed: 0', 'Clothing ID', 'Age', 'Title', 'Review Text', 'Rating', 'Recommended IND', 'Positive Feedback Count', 'Division Name', 'Department Nam
e', 'Class Name']
```

	Unnamed: 0	Clothing ID	Age	Title	Review Text	Rating	Recommended IND	Positive Feedback Count	Division Name	Department Name	Class Name
0	0	767	33	NaN	Absolutely wonderful - silky and sexy and comf...	4	1	0	Initmates	Intimate	Intimates
1	1	1080	34	NaN	Love this dress! it's sooo pretty. i happene...	5	1	4	General	Dresses	Dresses
2	2	1077	60	Some major design flaws	I had such high hopes for this dress and reall...	3	0	0	General	Dresses	Dresses
3	3	1049	50	My favorite buy!	I love, love, love this jumpsuit. it's fun, fl...	5	1	0	General Petite	Bottoms	Pants
4	4	847	47	Flattering shirt	This shirt is very flattering to all due to th...	5	1	6	General	Tops	Blouses

Hình 6.5 – Comments df.head()/columns

Nhận xét: Nhãn Recommended IND bị lệch về lớp 1. Việc stratify khi split giúp duy trì tỷ lệ lớp tương đối ổn định giữa train, validation và test.

6.3.2. TOKENIZATION VÀ EMBEDDING

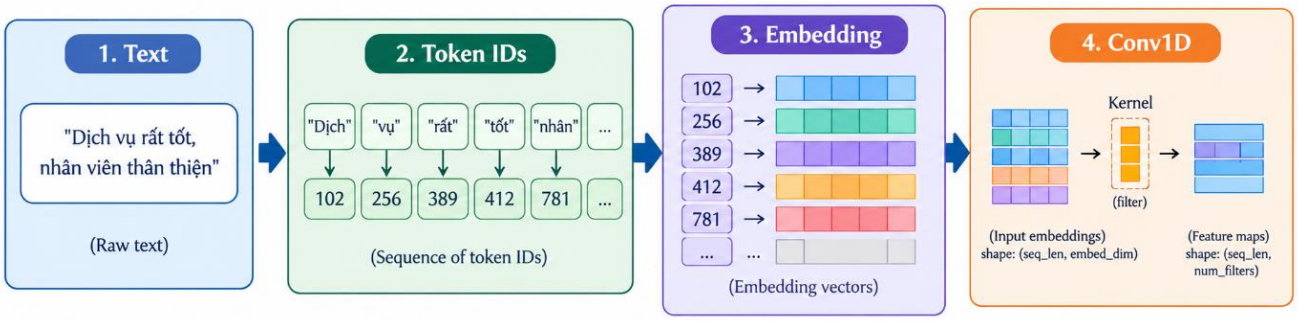
Khác với tabular, text cần được biểu diễn thành chuỗi token. Notebook chuyển text về lowercase rồi tách bằng whitespace. Từ điển được xây dựng từ training set, giới hạn tối đa 10,000 token; hai token đặc biệt là <PAD> và <UNK>.

Mỗi review được encode thành chuỗi dài cố định 100 token. Review dài hơn được cắt, review ngắn hơn được padding bằng <PAD>. Cách làm này tạo tensor input có shape (N,100).

Tham số	**Giá trị**

Max vocabulary	10,000
Embedding dimension	64
Max sequence length	100
Train shape	(15,848, 100)
Validation shape	(3,396, 100)
Test shape	(3,397, 100)

Trong CNN, embedding biến mỗi token ID thành vector 64 chiều. Tensor sau embedding có dạng (batch, 100, 64), sau đó transpose thành dạng phù hợp Conv1D.



Hình 6.6 – Pipeline Text → Token IDs → Embedding → Conv1D

Nhận xét: Embedding là cầu nối giữa token rời rạc và biểu diễn liên tục. Sau embedding, Conv1D có thể tìm các pattern cục bộ trong chuỗi token.

6.3.3. PYTORCH CNN

CommentsCNN gồm Embedding, hai Conv1D 64 channel với kernel size 3, ReLU và AdaptiveMaxPool1d(1), sau đó Linear(64,1). Đây là biến thể phù hợp với định hướng tokens → embedding → Conv1D → ReLU → pooling → dense.

Model dùng BCEWithLogitsLoss và Adam với learning rate 0.001. Sau 5 epoch, training loss giảm từ 0.4078 xuống 0.1470.

Epoch	**Loss**
1	0.4078
2	0.3036
3	0.2413
4	0.1930
5	0.1470

Training time là 18.95 giây và Test Accuracy đạt 0.8805.

Nhận xét: Loss giảm rõ rệt và accuracy đạt 88.05%. CNN có thể học local linguistic patterns từ embedding, nhưng kết quả vẫn phụ thuộc mạnh vào tokenization, vocabulary và độ dài sequence.

6.4. TENSORFLOW – DIABETES

TensorFlow/Keras được dùng để tái hiện ý tưởng Conv1D cho Diabetes. Model gồm Conv1D 16, Conv1D 16, MaxPooling1D, Flatten và Dense sigmoid.

Epoch	Train Acc	Val Acc	Val Loss
1	0.7989	0.7478	0.4937
2	0.7914	0.7391	0.4876
3	0.8101	0.7478	0.4893
4	0.8119	0.7478	0.4791
5	0.8194	0.7478	0.4792

Test Accuracy TensorFlow là 0.8276 và training time là 1.18 giây.

```
tf_diabetes_model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Eve
used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 8, 16)	64
conv1d_1 (Conv1D)	(None, 8, 16)	784
max_pooling1d (MaxPooling1D)	(None, 4, 16)	0
flatten (Flatten)	(None, 64)	0
dense (Dense)	(None, 1)	65

Total params: 913 (3.57 KB)

Trainable params: 913 (3.57 KB)

Non-trainable params: 0 (0.00 B)

Hình 6.7 – TensorFlow Diabetes model summary

Nhận xét: TensorFlow đạt accuracy cao hơn lần chạy PyTorch trên Diabetes (0.8276 so với 0.6983). Với dataset nhỏ, chênh lệch training dynamics có thể rõ hơn. Kết quả này cần được hiểu như một lần chạy thực nghiệm cụ thể, không phải kết luận framework nào luôn tốt hơn.

6.5. TENSORFLOW – HOUSE PRICE

TensorFlow House Price CNN dùng Conv1D 16, Conv1D 16, MaxPooling1D, GlobalAveragePooling1D và Dense(1). GlobalAveragePooling1D giúp rút representation 133 chiều về 16 giá trị trước regression output.

Metric	PyTorch	TensorFlow
MAE	1.6100	1.6016
RMSE	1.9684	1.9611
R^2	0.1905	0.1965
Training time (s)	6.67	18.88

TensorFlow đạt MAE và RMSE thấp hơn một chút, đồng thời R^2 cao hơn một chút. Tuy nhiên thời gian training dài hơn.

```
tf_house_model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 133, 16)	64
conv1d_3 (Conv1D)	(None, 133, 16)	784
max_pooling1d_1 (MaxPooling1D)	(None, 66, 16)	0
global_average_pooling1d (GlobalAveragePooling1D)	(None, 16)	0
dense_1 (Dense)	(None, 1)	17

```
Total params: 865 (3.38 KB)

Trainable params: 865 (3.38 KB)

Non-trainable params: 0 (0.00 B)
```

Hình 6.8 – TensorFlow House Price model summary

Nhận xét: Hai framework cho kết quả rất gần nhau về chất lượng dự đoán. Chênh lệch nhỏ này cho thấy cùng kiến trúc có thể được triển khai theo hai framework mà vẫn giữ đặc tính chung của mô hình.

6.6. TENSORFLOW – COMMENTS

TensorFlow Comments CNN dùng Embedding với vocabulary 10,000 và embedding dimension 64, hai Conv1D 64 channel, GlobalMaxPooling1D và Dense sigmoid.

Epoch	Train Acc	Val Acc	Val Loss
1	0.8262	0.8486	0.3336
2	0.8868	0.8763	0.2836
3	0.9249	0.8743	0.2951
4	0.9522	0.8722	0.3607
5	0.9731	0.8698	0.4022

Test Accuracy TensorFlow là 0.8717 và training time là 21.66 giây.

```
tf_comments_model.summary()
```

```
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 64)	640,000
conv1d_4 (Conv1D)	(None, 100, 64)	12,352
conv1d_5 (Conv1D)	(None, 100, 64)	12,352
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0
dense_2 (Dense)	(None, 1)	65

```
Total params: 664,769 (2.54 MB)

Trainable params: 664,769 (2.54 MB)

Non-trainable params: 0 (0.00 B)
```

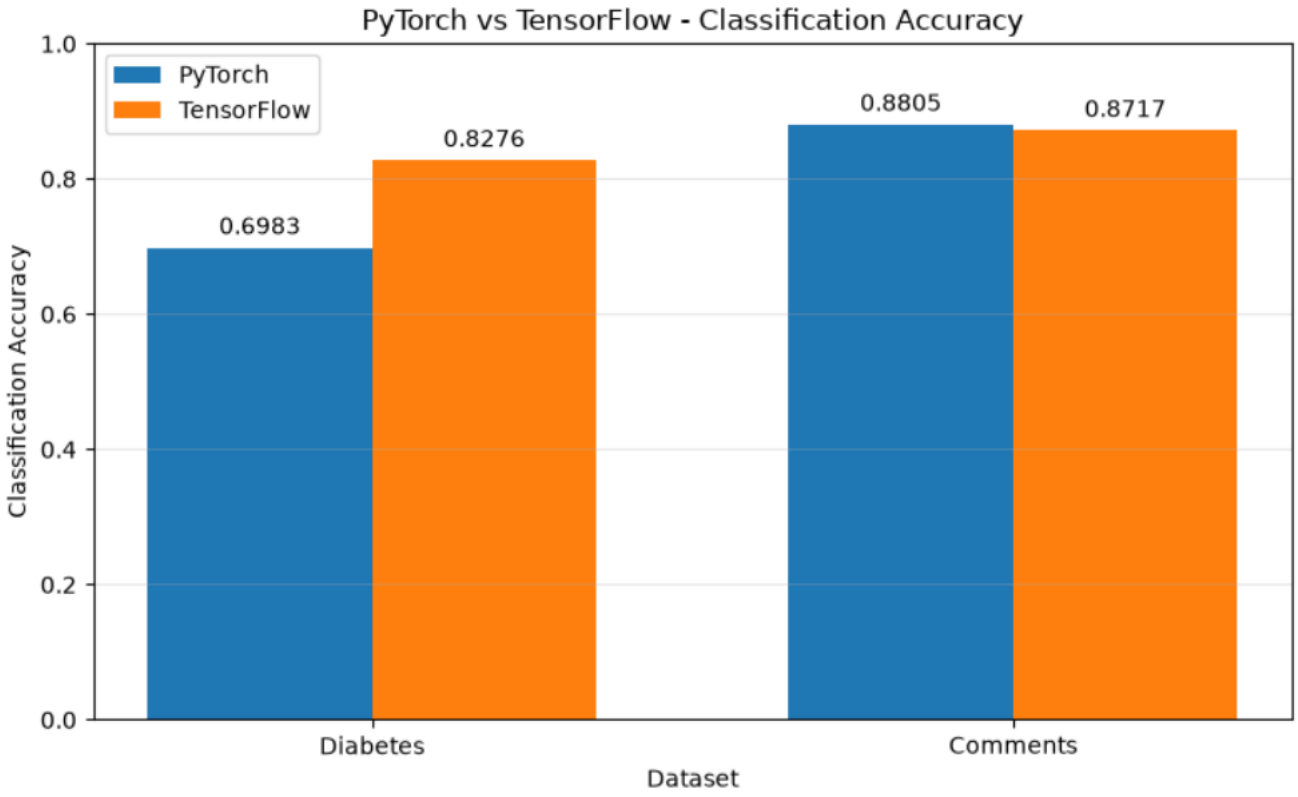
Hình 6.9 – TensorFlow Comments model summary

Nhận xét: Training accuracy tiếp tục tăng nhưng validation accuracy giảm nhẹ sau epoch 2 và validation loss tăng, cho thấy dấu hiệu overfitting. Test accuracy 0.8717 thấp hơn PyTorch 0.8805 trong lần chạy này.

6.7. SO SÁNH PYTORCH VÀ TENSORFLOW TRÊN BA DATASET 1D

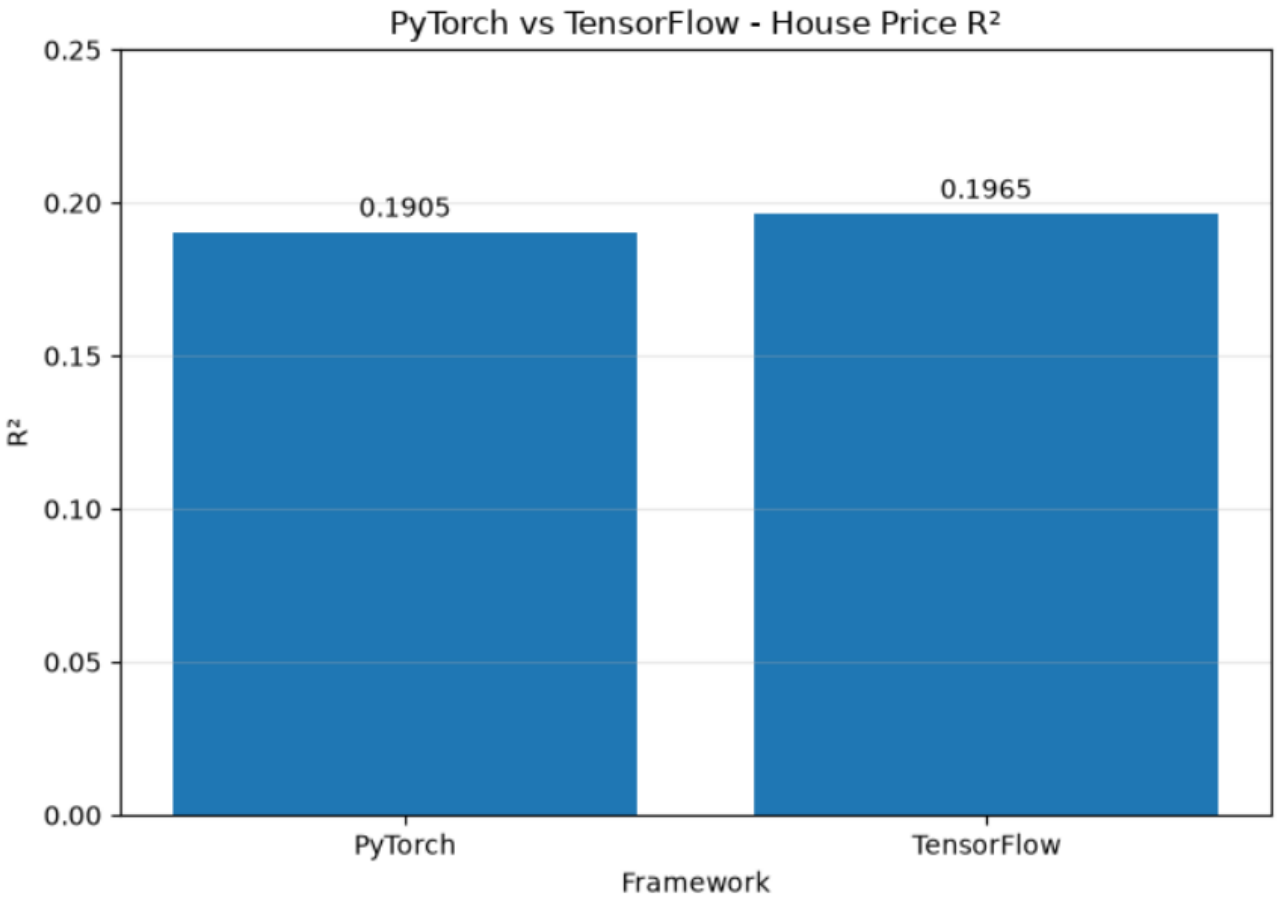
Dataset	Framework	Metric	Score	Time (s)
Diabetes	PyTorch	Accuracy	0.6983	0.50
Diabetes	TensorFlow	Accuracy	0.8276	1.18
House Price	PyTorch	R^2	0.1905	6.67
House Price	TensorFlow	R^2	0.1965	18.88
Comments	PyTorch	Accuracy	0.8805	18.95
Comments	TensorFlow	Accuracy	0.8717	21.66

Bảng cho thấy không có một framework duy nhất thắng trên tất cả dataset. TensorFlow tốt hơn trong lần chạy Diabetes và House Price, trong khi PyTorch tốt hơn trên Comments. Training time cũng phụ thuộc dataset và framework.



Hình 6.10 – So sánh Classification Accuracy của PyTorch và TensorFlow trên Diabetes và Comments.

Nhận xét: Kết quả cho thấy độ chính xác của hai framework thay đổi tùy theo bộ dữ liệu. Trên Diabetes, TensorFlow đạt 0.8276 trong khi PyTorch đạt 0.6983. Trên Comments, PyTorch đạt 0.8805 và TensorFlow đạt 0.8717.



Hình 6.11 – So sánh R^2 của PyTorch và TensorFlow trên bài toán dự đoán giá nhà.

Nhận xét: Đối với bài toán House Price, mô hình PyTorch đạt $R^2 = 0.1905$, trong khi mô hình TensorFlow đạt $R^2 = 0.1965$. Hai kết quả khá gần nhau, cho thấy hiệu năng của hai mô hình trên cùng bài toán có sự tương đồng.

So sánh framework cần được đặt trong cùng kiến trúc, cùng preprocessing và cùng budget training. Các kết quả ở đây phản ánh đúng những lần chạy trong notebook.

7. GIẢI THÍCH CODE NOTEBOOK

7.1. IMPORT VÀ DATA PIPELINE

Các cell import chuẩn bị toàn bộ thư viện cần thiết. NumPy và Pandas xử lý dữ liệu; Matplotlib phục vụ trực quan hóa; Scikit-learn thực hiện train_test_split, preprocessing và metrics; PyTorch cung cấp tensor, layer và optimizer; Torchvision hỗ trợ ImageFolder; TensorFlow/Keras dùng cho phần framework thứ hai.

Các cell load dataset sử dụng đường dẫn tương đối, giúp notebook hoạt động khi đặt đúng cấu trúc Assignment_04. Sau khi đọc dữ liệu, notebook in shape, columns hoặc class để xác nhận dữ liệu đã được load đúng.

Ở tabular, ColumnTransformer kết hợp numeric pipeline và categorical pipeline. Ở text, vocabulary được xây dựng từ training data trước khi encode validation/test, giúp tránh dùng thông tin vocabulary của test set trong quá trình học.

```
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics import (
    accuracy_score,
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

print("PyTorch:", torch.__version__)
```

Hình 7.1 – Import/load data

Data pipeline là phần quyết định input representation. CNN chỉ có ý nghĩa khi tensor đầu vào đúng shape và target đúng task.

7.2. MODEL DEFINITION VÀ FORWARD

Trong PyTorch, mỗi model kế thừa nn.Module. Hàm __init__ khai báo các layer, còn forward mô tả luồng dữ liệu. Sequential được dùng khi kiến trúc đi tuần tự.

BasicCNN ảnh có features gồm Conv2D, ReLU và MaxPool lặp lại; classifier gồm Flatten và Linear. DiabetesCNN và HousePriceCNN dùng Conv1D vì input được biểu diễn như sequence. CommentsCNN có thêm Embedding trước Conv1D.

Một chi tiết quan trọng là shape convention. Tensor ảnh PyTorch có dạng batch × channel × height × width. Với Conv1D, dạng chuẩn là batch × channel × length. Vì dữ liệu tabular được lưu dưới dạng samples × features × 1, code transpose để đưa channel lên trước.

```python
class DiabetesCNN(nn.Module):
    def __init__(self, num_features):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv1d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.Conv1d(16, 16, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.MaxPool1d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * (num_features // 2), 1)
        )

    def forward(self, x):
        x = x.transpose(1, 2)
        x = self.features(x)
        return self.classifier(x).squeeze(1)
```

Hình 7.2 – Khai báo class Diabetes

```python
class CommentsCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim=64):
        super().__init__()

        self.embedding = nn.Embedding(
            vocab_size,
            embedding_dim,
            padding_idx=0
        )

        self.features = nn.Sequential(
            nn.Conv1d(
                embedding_dim,
                64,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU(),
            nn.Conv1d(
                64,
                64,
                kernel_size=3,
                padding=1
            ),
            nn.ReLU(),
            nn.AdaptiveMaxPool1d(1)
        )

        self.classifier = nn.Linear(64, 1)

    def forward(self, x):
        x = self.embedding(x)
        x = x.transpose(1, 2)
        x = self.features(x)
        x = x.squeeze(-1)
        return self.classifier(x).squeeze(1)
```

Hình 7.3 – Khai báo class Comments

7.3. TRAINING LOOP

Training loop PyTorch trong notebook gồm model.train(), lấy batch từ DataLoader, đưa batch lên device, zero_grad, forward, tính loss, backward và optimizer.step. Đây là cách triển khai trực tiếp của chu trình forward → loss → backward → update.

Hàm train_cnn được dùng cho Diabetes và House Price. Với classification, criterion là BCEWithLogitsLoss; với regression, criterion là MSELoss. Comments sử dụng loop riêng do input token cần dtype long cho Embedding.

TensorFlow sử dụng model.compile() và model.fit(), trong đó optimizer, loss và metrics được khai báo trước. History trả về giúp lấy train/validation accuracy và loss để vẽ biểu đồ.

```python
def train_cnn(model, train_loader, val_loader, epochs=5, task="classification"):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    if task == "classification":
        criterion = nn.BCEWithLogitsLoss()
    else:
        criterion = nn.MSELoss()

    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

    start_time = time.time()

    for epoch in range(epochs):
        model.train()
        train_loss = 0

        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()

            output = model(X_batch)
            loss = criterion(output, y_batch)

            loss.backward()
            optimizer.step()

            train_loss += loss.item()

        print(
            f"Epoch {epoch + 1}/{epochs} "
            f"- Loss: {train_loss / len(train_loader):.4f}"
        )

    return time.time() - start_time
```

Hình 7.4 – Hàm train dùng cho Diabetes và House Price

```
start_time = time.time()

history_tf_diabetes = tf_diabetes_model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=5,
    batch_size=32,
    verbose=1
)

tf_diabetes_time = time.time() - start_time

tf_diabetes_loss, tf_diabetes_accuracy = tf_diabetes_model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print(f"Diabetes TensorFlow Accuracy: {tf_diabetes_accuracy:.4f}")
print(f"Training time: {tf_diabetes_time:.2f} seconds")
```

Hình 7.5 – TensorFlow model.fit()

Nhận xét: PyTorch cho thấy rõ từng bước tối ưu; TensorFlow/Keras rút gọn training loop. Đây chính là sự khác biệt implementation trong khi kiến trúc CNN vẫn tương đương.

7.4. EVALUATION

Classification evaluation dùng sigmoid hoặc model.evaluate tùy framework, sau đó tính accuracy. Với MNIST, Fashion-MNIST và CIFAR-10, classification_report được sử dụng để xem precision, recall và F1-score theo class.

House Price dùng evaluate_regression để lấy predictions và targets rồi tính MAE, RMSE và R^2. Đây là các metric phù hợp với regression và cũng là nhóm metric được định hướng trong bài học.

Việc tách evaluation khỏi training giúp đảm bảo test set chỉ được sử dụng để đánh giá cuối cùng, không dùng để cập nhật trọng số.

```python
def evaluate_classification(model, test_loader):
    device = next(model.parameters()).device

    model.eval()

    predictions = []
    targets = []

    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch = X_batch.to(device)

            output = torch.sigmoid(model(X_batch))
            pred = (output >= 0.5).int().cpu().numpy()

            predictions.extend(pred)
            targets.extend(y_batch.numpy())

    return accuracy_score(targets, predictions)


diabetes_accuracy = evaluate_classification(
    diabetes_model,
    diabetes_test_loader
)

print(f"Diabetes Test Accuracy: {diabetes_accuracy:.4f}")
```

Hình 7.6 – Code evaluate classification

```python
def evaluate_regression(model, test_loader):
    device = next(model.parameters()).device

    model.eval()

    predictions = []
    targets = []

    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch = X_batch.to(device)

            output = model(X_batch)

            predictions.extend(output.cpu().numpy())
            targets.extend(y_batch.numpy())

    mae = mean_absolute_error(targets, predictions)
    rmse = np.sqrt(mean_squared_error(targets, predictions))
    r2 = r2_score(targets, predictions)

    return mae, rmse, r2
```

Hình 7.7 – Code evaluate regression

Nhận xét: Evaluation phải dùng metric phù hợp task. Accuracy không thể thay thế trực tiếp cho MAE/RMSE/R^2 trong bài toán dự đoán giá.

8. TỔNG HỢP KẾT QUẢ TOÀN BỘ THỰC NGHIỆM

Kết quả Image CNN

Dataset	Input	Accuracy	Time (s)
MNIST	1×28×28	0.9850	137.146
Fashion-MNIST	1×28×28	0.9013	53.912
CIFAR-10	3×32×32	0.6477	800.702

Kết quả Tabular/Text CNN

Dataset	Framework	Metric	Score	Time (s)
Diabetes	PyTorch	Accuracy	0.6983	0.50
Diabetes	TensorFlow	Accuracy	0.8276	1.18

House Price	PyTorch	R^2	0.1905	6.67
House Price	TensorFlow	R^2	0.1965	18.88
Comments	PyTorch	Accuracy	0.8805	18.95
Comments	TensorFlow	Accuracy	0.8717	21.66

Không nên xếp hạng trực tiếp tất cả dataset theo một con số vì các metric và task khác nhau. Cần so sánh trong từng nhóm classification/regression và trong cùng dataset.

9. PHÂN TÍCH KẾT QUẢ VÀ BÀI HỌC

Thực nghiệm cho thấy CNN phát huy rất tốt trên MNIST với accuracy khoảng 98.5%. Khi chuyển sang Fashion-MNIST, accuracy giảm xuống khoảng 90.1%, còn CIFAR-10 chỉ đạt khoảng 64.8% với CNN cơ bản. Điều này phản ánh sự khác biệt về độ phức tạp của representation và nhiệm vụ.

Với tabular, Diabetes đạt 69.83% bằng PyTorch và 82.76% bằng TensorFlow trong lần chạy cụ thể. House Price đạt R^2 khoảng 0.19–0.20. Đây là kết quả vừa phải và cho thấy CNN 1D không tự động trở thành lựa chọn tối ưu cho dữ liệu bảng.

Với Comments, Embedding + Conv1D đạt 88.05% bằng PyTorch và 87.17% bằng TensorFlow. Đây là minh họa rõ ràng cho local pattern trong chuỗi token.

Một bài học quan trọng là architecture phải theo data representation. Ảnh dùng Conv2D vì có cấu trúc không gian hai chiều; tabular dùng Conv1D chỉ khi ta chấp nhận xem feature như một sequence; text dùng embedding để đưa token về không gian liên tục trước Conv1D.

Kết quả không chỉ trả lời model nào cao nhất mà cho thấy tại sao cùng một họ kiến trúc cần được điều chỉnh theo representation.

10. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

Các thực nghiệm được thiết kế theo mục tiêu học thuật và thời gian training giới hạn 5 epoch, do đó chưa tối ưu hyperparameter. CIFAR-10 đặc biệt cho thấy giới hạn của CNN cơ bản khi chưa dùng architecture hiện đại.

Ở House Price, representation tabular được tạo bằng preprocessing và one-hot encoding rồi đưa vào Conv1D. Cách này phù hợp để minh họa ý tưởng bài học nhưng không có nghĩa thứ tự feature luôn mang ý nghĩa ngữ nghĩa như thứ tự token trong text.

Ở Comments, tokenization sử dụng lowercase và split đơn giản, vocabulary giới hạn 10,000 và sequence length 100. Những lựa chọn này dễ triển khai nhưng có thể làm mất thông tin ngôn ngữ hoặc cắt review dài.

Hướng phát triển:

- Tăng số epoch và điều chỉnh learning rate.
- Thử regularization, dropout hoặc early stopping.
- Với CIFAR-10, thử CNN sâu hơn hoặc các kiến trúc hiện đại như ResNet-style.
- Với text, thử tokenizer tốt hơn và sequence length phù hợp.
- Với tabular, thử mô hình chuyên dụng cho dữ liệu bảng để làm baseline công bằng.
- Thực hiện nhiều random seeds để đánh giá độ ổn định.

Các hạn chế trên không làm mất giá trị bài thực hành; chúng giúp chỉ ra rõ ranh giới giữa CNN cơ bản dùng để học nguyên lý và một hệ thống Deep Learning tối ưu cho sản phẩm.

11. KẾT LUẬN

Assignment 04 đã triển khai CNN theo đúng chuỗi tư duy từ phép toán đến framework. Phần NumPy cho phép quan sát convolution, ReLU và pooling ở mức operation. PyTorch cho phép xây dựng model, autograd và training loop rõ ràng. TensorFlow/Keras cung cấp cách triển khai high-level với automatic differentiation và model.fit().

Ba dataset ảnh cho thấy CNN xử lý tốt những pattern không gian và hiệu quả phụ thuộc độ khó dữ liệu. Ba dataset 1D cho thấy Conv1D có thể được áp dụng cho feature sequence và token sequence, nhưng representation cần được thiết kế phù hợp.

Kết quả tốt nhất của nhóm thực nghiệm ảnh là MNIST với Test Accuracy 0.9850 đối với Basic CNN; Improved CNN đạt 0.9855. Trong phần 1D, Comments PyTorch đạt 0.8805 và TensorFlow đạt 0.8717; House Price đạt R^2 0.1905 và 0.1965 tương ứng; Diabetes đạt 0.6983 và 0.8276.

Qua bài thực hành, CNN được hiểu không phải là một hộp đen, mà là một cấu trúc gồm các phép biến đổi có mục tiêu: convolution học local patterns và chia sẻ trọng số, ReLU tạo phi tuyến, pooling giảm representation và Dense thực hiện nhiệm vụ cuối.

Thông điệp chính của Assignment 04 là: cùng một kiến trúc có thể có nhiều implementation, nhưng việc thiết kế representation phải xuất phát từ loại dữ liệu và task.

PHỤ LỤC – DATASET VÀ SOURCE CODE

Dataset

MNIST in CSV – Kaggle: https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

Fashion-MNIST – Kaggle: https://www.kaggle.com/datasets/zalando-research/fashionmnist

CIFAR-10 – Kaggle: https://www.kaggle.com/datasets/ayush1220/cifar10

Vietnam Housing Dataset 2024 – Kaggle: https://www.kaggle.com/datasets/nguyentiennhan/vietnam-housing-dataset-2024

Womens Clothing E-Commerce Reviews – Kaggle: https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

Diabetes – Kaggle: https://www.kaggle.com/datasets/jamaltariqcheema/pima-indians-diabetes-dataset/data

Source code

GitHub repository: https://github.com/tnd17/Assignment-04-PTHTTM-N05

Hai notebook chính: image_cnn_a04.ipynb và tabular_text_cnn_a04.ipynb.